

# Sorting Algorithms and their performance comparison using different parallelism techniques

Muhammad Ahsan

Department of Computer Science  
Virginia Commonwealth University, VCU  
Richmond, USA  
ahsanm5@vcu.edu

**Abstract**—This report presents different sorting algorithms and their performance evaluation from serial to parallel approach. Different parallelism techniques were implemented including multi-threading, Message Passing Interface (MPI) and CUDA GPU. The sorting algorithm being evaluated includes merge sort, quick sort and bitonic sort. The speedup of all these techniques were calculated and compared for variable number of threads and variable length of unsorted integer arrays (64K, 218K, 256K and 512K). From the comparison of these results it was concluded that the multi-threaded version of the algorithm outperforms MPI implementation in terms of speed up. But if the GPU version of the code is efficiently implemented it can outperform all, especially in the case of bitonic sort, since the algorithm is designed for parallel hardware.

**Index Terms**—HPDS, Multi-Threading, MPI, CUDA, Sorting

## I. INTRODUCTION

This report summarizes the results achieved for improving the serialized version of different sorting algorithms including merge sort, quick sort and bitonic sort having a run-time of  $O(n \log n)$ ,  $O(n \log n)$  and  $O(n \log^2 n)$  respectively.

Merge Sort [1] is a divide and conquer algorithm. The algorithm recursively divides the input array into two unsorted parts and after sorting merges the two halves. The steps of the algorithm are:

- Step 1: Find Middle of the input array
- Step 2: Call merge sort for first half
- Step 3: Call merge sort for second half
- Step 4: Merge both sorted subarrays

The merge sort function is called recursively till the size of the array is just 1 after which the merge function is called to merge the arrays in the bottom up fashion.

Quick Sort [2] like merge sort is also a divide and conquer algorithm. The algorithm picks a pivot element and partitions the input array around the pivot point. The partition function puts the pivot element at the correct sorted position in the array and puts all the elements smaller than pivot to the left and greater and equal to the right of the pivot value. The pivot can be selected as either random value, starting value, end value or the median of the array.

The quick sort algorithm first calls the partition function and divides the array at the pivot sorted index. After which it recursively calls the quick sort function till single value is reached.

Bitonic Sort [3] is a classic parallel sorting algorithm, i.e. this algorithm is feasible for sorting in parallel hardware. The input number of elements should be order of  $2^n$  for bitonic sort. The bitonic sort works by creating a bitonic sequence, i.e. one half of the array is ordered in ascending and second half in descending order. The bitonic merge then combines the array in the bottom up fashion to yield a sorted output array.

Multi-threading [4] is a program model that gives flexibility to execute code in parallel using multiple threads. These threads share the same memory space using shared process resources. The operating system scheduling algorithms are used to schedule and control these threads.

Message Passing Interface (MPI) [5] designed for distributed and parallel computing architectures. Multiple processes use send/receive functions to communicate in a distributed or parallel environment.

GPU provides higher instruction and memory bandwidth than CPU. CUDA is a computing architecture designed to facilitate the development of parallel programs [6]. CUDA supports GPU programming to solve much complex problems more efficiently than CPU.

The rest of the report is organized as follows, Section II details the implementation technique and the approach used for parallelization, Section III details the speed up achieved for each parallelization approach on variable size of input array, Section IV discusses the results achieved from the experimentation and in the end conclusion is presented.

## II. APPROACH USED FOR PARALLELIZATION

This section details the approach used for parallelization using multi-threaded, MPI and CUDA.

### A. Multi-Threading

The generic procedure for implementing the multi-threaded program for merge-sort, quick-sort and bitonic-sort is the same. We used the C++ thread library for the purpose. We divide the input array with the total number of running threads to get the length of data to be processed by each thread. The formula used for the purpose is shown in equation below;

$$nthreads = (N + numThreads - 1) / numThreads \quad (1)$$

where  $N$  is the number of elements in the array,  $nthreads$  is the number of elements per thread and  $numThreads$  are the total number of running threads.

For merge-sort using multi-threading algorithm 1 takes unsorted input array and outputs the sorted array. Each threads sorts its portion of the array and once they are done the sorted partial arrays are then merged as explained in the algorithm.

---

**Algorithm 1** Multi-Threading algorithm for merge-sort

---

**Require:**  $InputArr, N, numThreads, nthreads$

**Ensure:**  $SortedArr$

```

 $M \leftarrow numThreads$ 
while  $M \neq 0$  do
     $threads[m] \leftarrow thread(quickSort, InputArr, m * nthreads, ((m + 1) * nthreads) - 1)$ 
     $M \leftarrow M - 1$ 
end while
for  $j \leftarrow 1; j < numThreads; j* = 2$  do
     $irange \leftarrow irange/2;$ 
     $stride \leftarrow nthreads * 2 * j;$ 
    for  $i \leftarrow 0; i < irange; i++$  do
         $startIndex \leftarrow i * stride;$ 
         $midIndex \leftarrow startIndex + stride/2 - 1;$ 
         $lastIndex \leftarrow startIndex + stride - 1;$ 
         $mergeThreads[i] \leftarrow thread(mergeArrays, InputArr, startIndex, midIndex, lastIndex);$ 
    end for
end for

```

---

In the end after all the threads finishes and joined to the main routine the run-time is calculated. The algorithm for quick-sort and bionic-sort follows the same procedure.

### B. MPI

For MPI version we used `scatterv` and `gatherv` functions. The input array is generated in the process with rank 0, after which `scatterv` function is used to scatter portion of input array to each process, where the number of elements per process is calculated using the same formula as in multi-threading. Once each process has its portion of array it will sort it using one of the sorting technique. After sorting, the arrays are then gathered using `gatherv` function. The `Gatherv` function gathers all the partially sorted arrays from each process. The resultant received array in process rank 0 is then processed again to merge the sorted sub arrays, using one of the sorting technique. Once the sorted array is received the time is calculated and all the processes are terminated. The bitonic sort using MPI is given in algorithm 2

### C. CUDA

For programming the CUDA version of merge-sort following steps were involved

- First we create and allocate our unsorted input array in host memory.
- Then allocate device memory using `cudaMalloc` function.

---

**Algorithm 2** Multi-Threading algorithm for bitonic-sort

---

**Require:**  $InputArr, N, numprocess, nprocess, rank, dir$

**Ensure:**  $sortedArr$

```

 $M \leftarrow numThreads$ 
scatterv  $InputArr$ 
while  $M \neq 0$  do
     $threads[m] \leftarrow thread(bitonicSort, Arr[rank], 0, nprocess, dir)$ 
     $M \leftarrow M - 1$ 
end while
gatherv  $Arr[rank]$ 
for  $j \leftarrow 1; j < numprocess; j* = 2$  do
     $irange \leftarrow irange/2;$ 
     $stride \leftarrow nthreads * 2 * j;$ 
    for  $i \leftarrow 0; i < irange; i++$  do
         $startIndex \leftarrow i * stride;$ 
         $midIndex \leftarrow startIndex + stride/2 - 1;$ 
         $lastIndex \leftarrow startIndex + stride - 1;$ 
         $bitonicMerge(sortedArr, i * stride, stride, dir);$ 
    end for
end for

```

---

- Copy unsorted input array from host to device using `cudaMemcpy` function for *HostToDevice*.
- Next the Kernel is configured for our current case we have used 1D configuration wherein `threadsPerBlock` are fixed to 1024.
- Each thread in Kernel executes and sorts the portion of the arrays.
- After the kernel finish execution the partial sorted array is copied to host using `cudaMemcpy` function for *deviceToHost*.
- After that sorted sub arrays are then merged together to output a fully sorted array.
- In the end time is measured and the device and host memory are set free.

For programming the CUDA version of quick-sort, dynamic-parallelism feature of cuda is used. In dunamic-parallelism a parent grid launches kernels called child grids. This feature is used in quick-sort as follows.

- Run a kernel with 1 grid and number of threads equals 1.
- Run partitioner in the kernel and get index.
- Recursively create a new stream for each left array and right array partitioned at index.
- Copy sorted array to host memory.
- Measure the time and free allocated device and host memory.

For bitonic sort we launches our kernel with the maximum of 1024 threads. Each thread is then referred with a single unsorted array index. The array is then copied to the shared memory of the device. After which the bitonic sort is performed. The stide for each iteration is calculated along with the relative position after which the elements are swapped based on the direction. The Cuda version of the bitonic sort is not correct and producing erroneous results.

### III. PERFORMANCE EVALUATION

For the purpose of experimentation 4 sizes of arrays of size 64K, 128K, 256K and 512K were considered. For measuring the performance of the parallelization technique the KPI used is speedup which is defined as

$$Speedup = Serial\ Time / Parallel\ Time \quad (2)$$

$$Speedup = T_s / T_p \quad (3)$$

The second KPI we used is estimating the portion of parallel code which is calculated using the following formula

$$P - estimated = \frac{(1/Speedup) - 1}{(1/N) - 1} \quad (4)$$

Where  $N$  is the number of processes/threads. For running the implemented code the machine used is hosted at maple.vcu.edu having 28 processors.

The speed-up for each of the sizes of input arrays is calculated using 1, 2, 4, 8, 16, 32, 64 and 128 number of threads.

### IV. RESULTS AND DISCUSSION

Table I and Table II lists merge-sort execution time, speedup and estimation of parallel code for 64K, 128K, 256K and 512K size of arrays using 1, 2, 4, 8, 16, 32, 64 and 128 threads and process respectively. Using multi-threaded approach the maximum speed up achieved is 9.53, and using MPI the maximum speed up attained is 4.8.

Table III and Table IV lists quick-sort execution time, speedup and estimation of parallel code for 64K, 128K, 256K and 512K size of arrays using 1, 2, 4, 8, 16, 32, 64 and 128 threads and process respectively. Using multi-threaded approach the maximum speed up achieved is 4.72, and the maximum speed up attained using MPI is 2.73.

Table V and VI lists bitonic-sort execution time, speedup and estimation of parallel code for 64K, 128K, 256K and 512K size of arrays using 1, 2, 4, 8, 16, 32, 64 and 128 threads and process respectively. Using multi-threaded approach the maximum speed up achieved is 4.99, and using MPI the maximum speed up attained is 2.47.

Table VII and Table VIII shows the time and speed-ups for the CUDA version of the code for merge-sort and quick-sort respectively. The time measured for the GPU version for merge-sort is way higher than serial version. The reasons for which is some portion of time added to copy array to and from device memory. But the main reason for so much time is in-efficient technique used to code merge-sort. The time measured for quick-sort is relatively smaller but still higher than serial version the reasons are the same.

Figure 1, 2 shows the speed up trend for merge sort using multi-threading and MPI respectively for variable size of input array. Figure 3, 4 shows speed up trend for quick sort and Figure 5, 6 shows speed up trend for bitonic sort for multi-threading and MPI version of the algorithms.

Merge Sort Multi-Threading Speedup

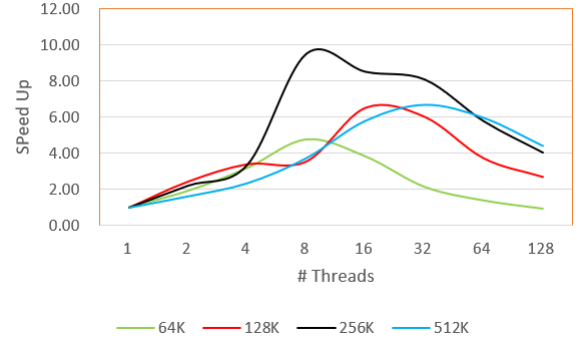


Fig. 1. Speedup Comparison Plot for different number of Threads and array sizes

Merge Sort MPI Speedup

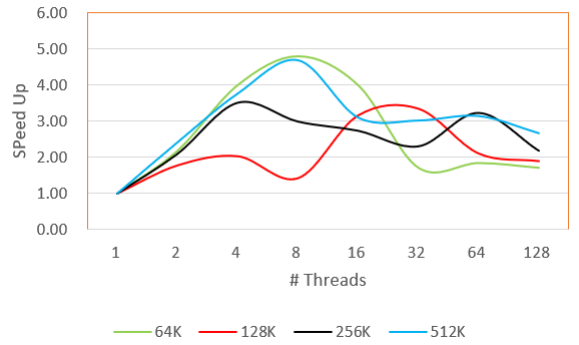


Fig. 2. Speedup Comparison Plot for different number of Threads and array sizes

### V. CONCLUSION

In conclusion we have presented the sequential, multi-threaded, MPI and CUDA version implementation of merge-sort, quick-sort and bitonic-sort. The algorithms were tested for variable length of input array i.e. 64K, 128K, 256K and 512K. For the current application scenario the multi-threading performs better than MPI in terms of speedup, but the GPU version of the code if implemented efficiently could

Quick Sort Multi-Threading Speedup

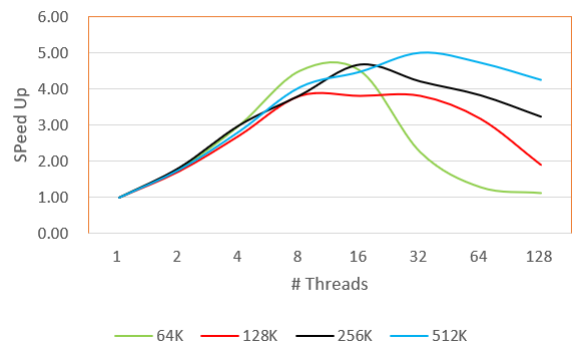


Fig. 3. Speedup Comparison Plot for different number of Threads and array sizes

TABLE I  
TIME, SPEED-UP AND  $P_{est}$  FOR MERGE SORT USING MULTI-THREADING

| Number of Threads | 64K       |         |           | 128K      |         |           | 256K      |         |           | 512K      |         |           |
|-------------------|-----------|---------|-----------|-----------|---------|-----------|-----------|---------|-----------|-----------|---------|-----------|
|                   | Time (ms) | Speedup | $P_{est}$ | Time (ms) | Speedup | $P_{est}$ | Time (ms) | Speedup | $P_{est}$ | Time (ms) | Speedup | $P_{est}$ |
| 1                 | 19        | 1       | 0         | 78        | 1       | 0         | 162       | 1       | 0         | 221       | 1       | 0         |
| 2                 | 10        | 1.9     | 0.95      | 32        | 2.44    | 1.18      | 73        | 2.22    | 1.1       | 136       | 1.63    | 0.77      |
| 4                 | 6         | 3.17    | 0.91      | 23        | 3.39    | 0.94      | 48        | 3.38    | 0.94      | 94        | 2.35    | 0.77      |
| 8                 | 4         | 4.75    | 0.9       | 22        | 3.55    | 0.82      | 17        | 9.53    | 1.02      | 59        | 3.75    | 0.84      |
| 16                | 5         | 3.8     | 0.79      | 12        | 6.5     | 0.9       | 19        | 8.53    | 0.94      | 38        | 5.82    | 0.88      |
| 32                | 9         | 2.11    | 0.54      | 13        | 6       | 0.86      | 20        | 8.1     | 0.9       | 33        | 6.7     | 0.88      |
| 64                | 14        | 1.36    | 0.27      | 21        | 3.71    | 0.74      | 28        | 5.79    | 0.84      | 37        | 5.97    | 0.85      |
| 128               | 21        | 0.9     | -0.11     | 29        | 2.69    | 0.63      | 40        | 4.05    | 0.76      | 50        | 4.42    | 0.78      |

TABLE II  
TIME, SPEED-UP AND  $P_{est}$  FOR MERGE SORT USING MPI

| Number of Threads | 64K       |         |           | 128K      |         |           | 256K      |         |           | 512K      |         |           |
|-------------------|-----------|---------|-----------|-----------|---------|-----------|-----------|---------|-----------|-----------|---------|-----------|
|                   | Time (ms) | Speedup | $P_{est}$ | Time (ms) | Speedup | $P_{est}$ | Time (ms) | Speedup | $P_{est}$ | Time (ms) | Speedup | $P_{est}$ |
| 1                 | 24        | 1       | 0         | 57        | 1       | 0         | 161       | 1       | 0         | 229       | 1       | 0         |
| 2                 | 11        | 2.18    | 1.08      | 32        | 1.78    | 0.88      | 77        | 2.09    | 1.04      | 95        | 2.41    | 1.17      |
| 4                 | 6         | 4       | 1         | 28        | 2.04    | 0.68      | 46        | 3.5     | 0.95      | 61        | 3.75    | 0.98      |
| 8                 | 5         | 4.8     | 0.9       | 40        | 1.43    | 0.34      | 54        | 2.98    | 0.76      | 49        | 4.67    | 0.9       |
| 16                | 6         | 4       | 0.8       | 18        | 3.17    | 0.73      | 59        | 2.73    | 0.68      | 74        | 3.09    | 0.72      |
| 32                | 14        | 1.71    | 0.43      | 17        | 3.35    | 0.72      | 70        | 2.3     | 0.58      | 76        | 3.01    | 0.69      |
| 64                | 13        | 1.85    | 0.47      | 27        | 2.11    | 0.53      | 50        | 3.22    | 0.7       | 73        | 3.14    | 0.69      |
| 128               | 14        | 1.71    | 0.42      | 30        | 1.9     | 0.48      | 74        | 2.18    | 0.54      | 86        | 2.66    | 0.63      |

TABLE III  
TIME, SPEED-UP AND  $P_{est}$  FOR QUICK SORT USING MULTI-THREADING

| Number of Threads | 64K       |         |           | 128K      |         |           | 256K      |         |           | 512K      |         |           |
|-------------------|-----------|---------|-----------|-----------|---------|-----------|-----------|---------|-----------|-----------|---------|-----------|
|                   | Time (ms) | Speedup | $P_{est}$ | Time (ms) | Speedup | $P_{est}$ | Time (ms) | Speedup | $P_{est}$ | Time (ms) | Speedup | $P_{est}$ |
| 1                 | 9         | 1       | 0         | 19        | 1       | 0         | 42        | 1       | 0         | 85        | 1       | 0         |
| 2                 | 5         | 1.8     | 0.89      | 11        | 1.73    | 0.84      | 23        | 1.83    | 0.9       | 48        | 1.77    | 0.87      |
| 4                 | 3         | 3       | 0.89      | 7         | 2.71    | 0.84      | 14        | 3       | 0.89      | 30        | 2.83    | 0.86      |
| 8                 | 2         | 4.5     | 0.89      | 5         | 3.8     | 0.84      | 11        | 3.82    | 0.84      | 21        | 4.05    | 0.86      |
| 16                | 2         | 4.5     | 0.83      | 5         | 3.8     | 0.79      | 9         | 4.67    | 0.84      | 19        | 4.47    | 0.83      |
| 32                | 4         | 2.25    | 0.57      | 5         | 3.8     | 0.76      | 10        | 4.2     | 0.79      | 17        | 5       | 0.83      |
| 64                | 7         | 1.29    | 0.23      | 6         | 3.17    | 0.7       | 11        | 3.82    | 0.75      | 18        | 4.72    | 0.8       |
| 128               | 8         | 1.13    | 0.11      | 10        | 1.9     | 0.48      | 13        | 3.23    | 0.7       | 20        | 4.25    | 0.77      |

TABLE IV  
TIME, SPEED-UP AND  $P_{est}$  FOR QUICK SORT USING MPI

| Number of Threads | 64K       |         |           | 128K      |         |           | 256K      |         |           | 512K      |         |           |
|-------------------|-----------|---------|-----------|-----------|---------|-----------|-----------|---------|-----------|-----------|---------|-----------|
|                   | Time (ms) | Speedup | $P_{est}$ | Time (ms) | Speedup | $P_{est}$ | Time (ms) | Speedup | $P_{est}$ | Time (ms) | Speedup | $P_{est}$ |
| 1                 | 9         | 1       | 0         | 19        | 1       | 0         | 41        | 1       | 0         | 85        | 1       | 0         |
| 2                 | 5         | 1.8     | 0.89      | 11        | 1.73    | 0.84      | 23        | 1.78    | 0.88      | 50        | 1.7     | 0.82      |
| 4                 | 4         | 2.25    | 0.74      | 8         | 2.38    | 0.77      | 22        | 1.86    | 0.62      | 43        | 1.98    | 0.66      |
| 8                 | 3         | 3       | 0.76      | 7         | 2.71    | 0.72      | 17        | 2.41    | 0.67      | 34        | 2.5     | 0.69      |
| 16                | 4         | 2.25    | 0.59      | 7         | 2.71    | 0.67      | 15        | 2.73    | 0.68      | 32        | 2.66    | 0.67      |
| 32                | 5         | 1.8     | 0.46      | 8         | 2.38    | 0.6       | 17        | 2.41    | 0.6       | 36        | 2.36    | 0.6       |
| 64                | 7         | 1.29    | 0.23      | 10        | 1.9     | 0.48      | 22        | 1.86    | 0.47      | 41        | 2.07    | 0.53      |
| 128               | 8         | 1.13    | 0.11      | 15        | 1.27    | 0.21      | 25        | 1.64    | 0.39      | 49        | 1.73    | 0.43      |

TABLE V  
TIME, SPEED-UP AND  $P_{est}$  FOR BITONIC SORT USING MULTI-THREADING

| Number of Threads | 64K       |         |           | 128K      |         |           | 256K      |         |           | 512K      |         |           |
|-------------------|-----------|---------|-----------|-----------|---------|-----------|-----------|---------|-----------|-----------|---------|-----------|
|                   | Time (ms) | Speedup | $P_{est}$ | Time (ms) | Speedup | $P_{est}$ | Time (ms) | Speedup | $P_{est}$ | Time (ms) | Speedup | $P_{est}$ |
| 1                 | 57        | 1       | 0         | 127       | 1       | 0         | 281       | 1       | 0         | 619       | 1       | 0         |
| 2                 | 32        | 1.78    | 0.88      | 73        | 1.74    | 0.85      | 158       | 1.78    | 0.88      | 336       | 1.84    | 0.91      |
| 4                 | 22        | 2.59    | 0.82      | 47        | 2.7     | 0.84      | 103       | 2.73    | 0.84      | 222       | 2.79    | 0.86      |
| 8                 | 16        | 3.56    | 0.82      | 35        | 3.63    | 0.83      | 75        | 3.75    | 0.84      | 159       | 3.89    | 0.85      |
| 16                | 15        | 3.8     | 0.79      | 32        | 3.97    | 0.8       | 68        | 4.13    | 0.81      | 143       | 4.33    | 0.82      |
| 32                | 15        | 3.8     | 0.76      | 32        | 3.97    | 0.77      | 61        | 4.61    | 0.81      | 126       | 4.91    | 0.82      |
| 64                | 17        | 3.35    | 0.71      | 30        | 4.23    | 0.78      | 66        | 4.26    | 0.78      | 132       | 4.69    | 0.8       |
| 128               | 22        | 2.59    | 0.62      | 33        | 3.85    | 0.75      | 65        | 4.32    | 0.77      | 124       | 4.99    | 0.81      |

TABLE VI  
TIME, SPEED-UP AND  $P_{est}$  FOR BITONIC SORT USING MPI

| Number of Threads | 64K       |         |           | 128K      |         |           | 256K      |         |           | 512K      |         |           |
|-------------------|-----------|---------|-----------|-----------|---------|-----------|-----------|---------|-----------|-----------|---------|-----------|
|                   | Time (ms) | Speedup | $P_{est}$ | Time (ms) | Speedup | $P_{est}$ | Time (ms) | Speedup | $P_{est}$ | Time (ms) | Speedup | $P_{est}$ |
| 1                 | 58        | 1       | 0         | 129       | 1       | 0         | 284       | 1       | 0         | 623       | 1       | 0         |
| 2                 | 32        | 1.81    | 0.9       | 71        | 1.82    | 0.9       | 157       | 1.81    | 0.89      | 341       | 1.83    | 0.91      |
| 4                 | 30        | 1.93    | 0.64      | 64        | 2.02    | 0.67      | 136       | 2.09    | 0.69      | 260       | 2.4     | 0.78      |
| 8                 | 26        | 2.23    | 0.63      | 57        | 2.26    | 0.64      | 126       | 2.25    | 0.64      | 252       | 2.47    | 0.68      |
| 16                | 27        | 2.15    | 0.57      | 59        | 2.19    | 0.58      | 125       | 2.27    | 0.6       | 265       | 2.35    | 0.61      |
| 32                | 31        | 1.87    | 0.48      | 66        | 1.95    | 0.5       | 141       | 2.01    | 0.52      | 293       | 2.13    | 0.55      |
| 64                | 37        | 1.57    | 0.37      | 77        | 1.68    | 0.41      | 159       | 1.79    | 0.45      | 337       | 1.85    | 0.47      |
| 128               | 43        | 1.35    | 0.26      | 92        | 1.4     | 0.29      | 187       | 1.52    | 0.34      | 389       | 1.6     | 0.38      |

TABLE VII  
TIME, SPEED-UP AND  $P_{est}$  FOR MERGE SORT USING CUDA

| Number of Threads | 64K       |         |           | 128K      |         |           | 256K      |         |           |
|-------------------|-----------|---------|-----------|-----------|---------|-----------|-----------|---------|-----------|
|                   | Time (ms) | Speedup | $P_{est}$ | Time (ms) | Speedup | $P_{est}$ | Time (ms) | Speedup | $P_{est}$ |
| 1                 | 8117.00   | 1.00    | 0.00      | 16708.00  | 1.00    | 0.00      | 34464.00  | 1.00    | 0.00      |
| 2                 | 4715.00   | 1.72    | 0.84      | 9719.00   | 1.72    | 0.84      | 20007.00  | 1.72    | 0.84      |
| 4                 | 3003.00   | 2.70    | 0.84      | 6158.00   | 2.71    | 0.84      | 12648.00  | 2.72    | 0.84      |
| 8                 | 2269.00   | 3.58    | 0.82      | 4616.00   | 3.62    | 0.83      | 9384.00   | 3.67    | 0.83      |
| 16                | 2078.00   | 3.91    | 0.79      | 4187.00   | 3.99    | 0.80      | 8456.00   | 4.08    | 0.80      |
| 32                | 2101.00   | 3.86    | 0.77      | 4217.00   | 3.96    | 0.77      | 8466.00   | 4.07    | 0.78      |
| 64                | 1994.00   | 4.07    | 0.77      | 3987.00   | 4.19    | 0.77      | 7983.00   | 4.32    | 0.78      |
| 128               | 2096.00   | 3.87    | 0.75      | 4174.00   | 4.00    | 0.76      | 8339.00   | 4.13    | 0.76      |

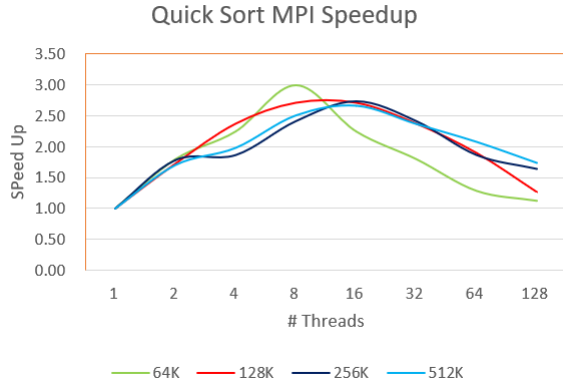


Fig. 4. Speedup Comparison Plot for different number of Threads and array sizes

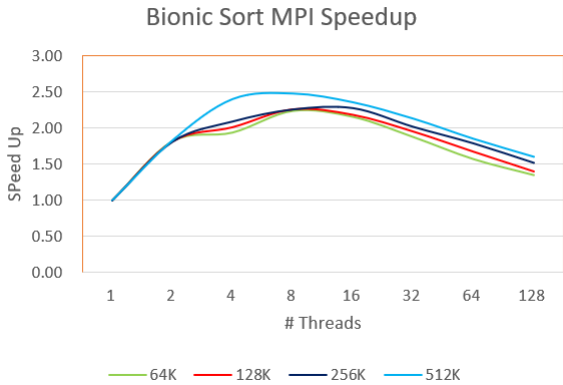


Fig. 5. Speedup Comparison Plot for different number of Threads and array sizes

Bionic Sort Multi-Threading Speedup

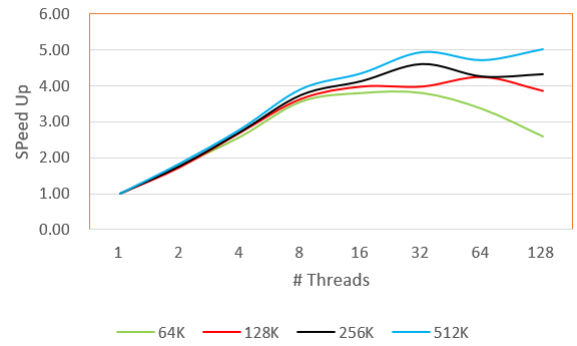


Fig. 6. Speedup Comparison Plot for different number of Threads and array sizes

TABLE VIII  
TIME FOR QUICK SORT USING CUDA

| Array length | Time (ms) |
|--------------|-----------|
| 64K          | 223.90    |
| 128          | 362.00    |
| 256          | 828.60    |
| 512          | 1703.00   |

outperform multi-threading especially in the case of bitonic sort which is designed for parallel hardware.

## REFERENCES

- [1] R. Cole, "Parallel merge sort," *SIAM Journal on Computing*, vol. 17, no. 4, pp. 770–785, 1988.
- [2] R. Sedgewick, "The analysis of quicksort programs," *Acta Informatica*, vol. 7, no. 4, pp. 327–355, 1977.
- [3] G. Bilardi and A. Nicolau, "Adaptive bitonic sorting: An optimal parallel algorithm for shared-memory machines," *SIAM Journal on Computing*, vol. 18, no. 2, pp. 216–228, 1989.

- [4] B. Lewis and D. J. Berg, *Multithreaded programming with Pthreads*. Prentice-Hall, Inc., 1998.
- [5] W. Gropp, W. D. Gropp, E. Lusk, A. Skjellum, and A. D. F. E. Lusk, *Using MPI: portable parallel programming with the message-passing interface*. MIT press, 1999, vol. 1.
- [6] J. Sanders and E. Kandrot, *CUDA by example: an introduction to general-purpose GPU programming*. Addison-Wesley Professional, 2010.